

GUIDE — Guide d'utilisation complet de PBLCA

Ce guide décrit le pipeline complet, étape par étape : configuration du cas d'étude par cartes déclaratives, exécution, analyse R, documentation générée et tests. Chaque section explique **ce que fait l'utilisateur**, **ce que fait le code**, et renvoie vers la documentation auto-générée (pdoc) et les fichiers sources.

Sommaire

1. Prérequis et installation
2. Architecture en trois couches (comprendre avant d'utiliser)
3. Étape 1 — Les deux cartes : ferme et étude
4. Étape 2 — Exécuter le pipeline
5. Étape 3 — Analyses R
6. Étape 4 — Tests et contrat Python → R
7. Étape 5 — Documentation auto-générée
8. Pour aller plus loin (niveau développeur)

1 — Prérequis et installation

Python (≥ 3.10) :

```
cd pblca
pip install -e ".[dev]"      # moteur + CLI + pytest + pdoc
```

R (≥ 4.0) pour les analyses et figures :

```
install.packages(c("jsonlite", "ggplot2"))
```

Vérification rapide :

```
pytest -q                # suite de tests (cf. Étape 4)
pblca --version           # CLI installé
pdoc pblca -o docs_html  # documentation HTML (cf. Étape 5)
Rscript -e "library(jsonlite)" # analyse R
```

2 — Architecture en trois couches (comprendre avant d'utiliser)

Le moteur est conforme ISO 14040/14044 : l'inventaire (kg de gaz émis) est

séparé de la caractérisation (conversion en kg CO₂e). Trois couches :

COUCHE 3 – IMPACTS	pblca/impacts.py	GWP100, GWP20, GWP* (Cain 2019)
COUCHE 2 – FLUX DE GAZ	pblca/gases.py	registre CH ₄ /CO ₂ /N ₂ O, traçabilité
COUCHE 1 – PROCESSUS	pblca/processes/	entérique, fumier, sol N ₂ O, carbone, achats, travaux mécaniques

Deux transverses :

- pblca/params.py — **ParameterSet** :

chaque paramètre porte sa valeur centrale, son incertitude (log-normale),

son unité et sa **référence bibliographique** (exigence ISO : aucune valeur non sourcée). Les facteurs GWP (table 7.15 AR6) sont des paramètres comme les autres : leur incertitude se propage dans le Monte-Carlo.

- `pblca/registry.py` — registre de modèles : pour chaque `*slot*` (ex. `enteric_ch4`), plusieurs variantes interchangeables (Tier-2 2006, Tier-2 2019, Tier-3 Mills, Tier-3 Sauvart 2011, mesures AHCS...), chacune avec sa référence. Interface universelle : un slot se change sans toucher au reste.

L'`LCAEngine` (`pblca/engine.py`)

orchestre : sélection des variantes → exécution des processus dans l'ordre des dépendances physiques (le N organique est connu AVANT le module sol) → remplissage du `GasLedger` → calcul des

impacts (`characterize`) → stockage

JSON. La sous-couche `pblca/mc.py` génère les tirages Monte-Carlo

(`ParameterSet.draw`) et perturbe

les rations mesurées ; les processus restent des évaluateurs purs.

Étape 1 — Les deux cartes : ferme et étude

Une étude = une paire de cartes TOML (déclaratif, rien à coder) :

la **carte ferme** porte les données, la **carte étude** porte les choix de modélisation. Toutes deux sont validées au chargement — une faute de frappe dans un slot, une variante ou une clé échoue immédiatement avec la liste des valeurs acceptées.

1.1 — La carte ferme : `cards/farms/ferme_20ha.toml`

Un fichier = une ferme, **source unique de vérité** des données :

- **le troupeau** — `[[farm.animals]]`, un bloc par lot :
 - `key`, `n_head` (effectif annuel moyen **déclaré** — la dynamique du troupeau, naissances comprises, appartient aux données de ferme et est découplée des achats), `days`, `bw_start/bw_end`, `diet_de`, `share_concentrate`, `system` ;
 - les **mesures on-farm** sur le groupe auquel elles appartiennent : `ch4_measured_ahcs` (GreenFeed), `dmi_measured` (feuilles de ration), `diet_om/diet_omd` (caractérisation INRA), avec leurs erreurs relatives ;
 - les **parcelles** — `[[farm.parcels]]` : culture, surface, travail du sol (`deep_tillage`), chaux, et deux **vecteurs d'événements datés** :
 - `[[farm.parcels.grazing]]` — pâturage : `group` (lot) + `entry/exit`

(dates). Plusieurs événements par parcelle et par an, sur prairie ou culture. La **part pâturage du fumier en est dérivée** (Σ jours / 365 par lot) et le **N déposé est routé vers la parcelle pâturée** ;

- `[[farm.parcels.synthetic_fertilisation]]` et `[[farm.parcels.organic_fertilisation]]` — fertilisations minérale et organique : type, date, n_kg **par apport** (plusieurs apports par an, sommés en taux annuels par parcelle). L'épandage de fumier est donc **explicite**, plus de redistribution automatique ;

- **la gestion du fumier en bâti** — `[farm.manure_split]` :

- `solid_storage / liquid_slurry` : le rapport fumier solide / lisier des excretions logées (la part pâturage n'y figure pas : dérivée des événements ; le lisier n'est pas encore implémenté — doit valoir 0.0, cf. Limites connues en 1.3) ;

- `manure_exported_fresh` : part exportée **avant** stockage (fumier frais vendu : aucune émission de stockage à la ferme — périmètre externe) ;

- `manure_exported_stored` : part exportée **après** stockage (compost, fumier vendu : les émissions de stockage restent à la ferme, seul l'épandage sort du périmètre) ;

- **les achats** — `[farm.purchases]` : concentrés, co-produits, engrais minéral ferme, et les jeunes animaux achetés (`n_calves_purchased × calf_purchased_bw_kg` → flux d'achat dérivé).

1.2 — La carte étude : `cards/studies/ferme_20ha.toml`

Un fichier = une étude reproductible : la ferme référencée, les modèles à tester, le plan Monte-Carlo, la sortie.

```
[study]
name = "ferme_20ha"           # préfixe des sim_id
farm = "ferme_20ha"          # carte ferme (cards/farms/), chemin .toml,
                              # ou table [farm] complète en ligne
main_enteric_variant = "tier2_fao_ym_modelled_ingestion" # référence de
                              # l'analyse appariée

[model_selection]             # modèles de référence – EXPLICITES pour
enteric_ch4 = "tier2_2006_modelled_ingestion" # chaque process
manure_ch4 = "ipcc_tier2"
manure_n2o = "ipcc_tier2"
soil_n2o = "ipcc_2019"
soil_carbon = "ipcc_stock_change"
purchases = "agribalyse_ecoinvent"
fieldwork = "fuel_ademe"

[variant_grid]                # sweeps mono-slot : chaque variante est
enteric_ch4 = [ ... ]          # exécutée à valeurs centrales puis en MC
manure_ch4 = ["ipcc_tier2", "tier3_eugene2019"]

[study.named_combinations]    # chaînes cohérentes (ex. INRA Tier-3)
```

```

inra_tier3 = { enteric_ch4 = "tier3_sauvant2011_modelled_ingestion",
               manure_ch4 = "tier3_eugene2019" }

[monte_carlo]
n_iterations = 500
seed = 2024

[datastore]
path = "results.json"

```

Section	Rôle
[study]	name ; farm (carte ferme / chemin / inline) ; named_combinations ; main_enteric_variant
[model_selection]	variantes du run central — explicites pour chaque process
[variant_grid]	sweeps mono-slot : valeurs centrales + Monte-Carlo par variante
[monte_carlo]	n_iterations, seed (reproductibilité)
[datastore]	path du JSON de résultats

Comment choisir `main_enteric_variant` : c'est la variante dont le GWP100 est décomposé (Étape 3) et le point zéro des figures d'effet-modèle.

Choisissez votre estimation de référence au niveau ferme.

Traçabilité : `results.json` embarque la carte étude elle-même (hachage SHA-256 + texte brut sous `card / card_source`) — chaque étude reste traçable et reproductible (ISO 14044 §4.5). Les variantes exigeant des mesures que la ferme ne possède pas sont exclues automatiquement, avec la raison (une carte est donc échangeable entre fermes).

Le cas d'étude reste aussi scriptable en Python (`CaseStudyConfig` dans `pblca/scenarios.py`), mais la voie recommandée est la paire de cartes.

1.3 — Limites connues du moteur

À connaître avant de décrire une ferme — ces limites sont volontairement verrouillées au chargement des cartes (fail-fast avec message explicite) plutôt que silencieusement approximées :

1. **La filière lisier n'est pas encore implémentée** (`liquid_slurry` doit valoir 0.0 dans `[farm.manure_split]`) : le moteur ne dispose pas de MCF/EF3 lisier **sourcés** (IPCC 2006/2019, table 10.17 pour le MCF lisier par climat, EF3 lisier), et la règle du dépôt interdit toute valeur non sourcée. Pour lever le verrou : fournir les paramètres sourcés dans `pblca/params.py` et la chaîne de traitement dans `pblca/processes/manure.py` (la structure est prête, il ne manque que les valeurs référencées).

2. **Les dates des événements sont stockées mais non consommées** : les pâturages et fertilisations sont datés en prévision d'un futur modèle de sol **dynamique** (pas de temps journalier), mais le moteur actuel

annualise (moyenne annuelle par parcelle). Les dates servent dès maintenant à la traçabilité et à la validation.

3. ****Le N déposé au pâturage est routé mais pas encore consommé par le carbone du sol**** : `n_excreta_grazing` (kg N/ha/an) alimente la traçabilité et exclut le double comptage **N■O**, mais le module carbone du sol ne l'utilise pas encore comme entrée de stock (prémices du point 2). Ces limites sont des choix explicites de traçabilité (aucune valeur approximative sans référence), pas des oublis.

Étape 2 — Exécuter le pipeline

```
pblca run cards/studies/ferme_20ha.toml # la voie déclarative
python run_case_study.py                # équivalent (délègue au CLI)
```

Le pipeline (`pblca/scenarios.py`, `run_case_study`) :

1. **Grille de scénarios** — une exécution à valeur centrale par combinaison de variantes ; les combinaisons non exécutables (mesures manquantes) sont exclues avec la raison.

2. **Monte-Carlo par variante** — un MC indépendant par variante de la grille (`mc_<cas>_<slot>_<variante>`) : statistiques (mean, sd, p5, p50, p95) des émissions entériques par lot, par tête/jour, du **CH■** fumier par système, et des impacts ferme.

3. **Comparaison appariée des rations** — à chaque itération, un seul tirage de paramètres, deux évaluations (équations IPCC vs rations mesurées) : la différence appariée isole l'effet du mode de ration.

4. **Grille appariée des variantes entériques** — le cœur de l'analyse de sensibilité :

```
LCAEngine.run_paired_variant_grid.
```

À chaque itération, **un seul tirage**, puis toutes les variantes évaluées avec ce même tirage : les colonnes sont directement comparables ligne à ligne — une différence entre deux colonnes d'une même ligne reflète le choix du modèle, pas le bruit d'échantillonnage.

Cohérence fumier → sol (automatique, ordre physique) : le module fumier dérive la part pâturage de chaque lot des événements de pâturage, route le N déposé vers les parcelles pâturées, calcule le fumier stocké disponible par différence (excretions logées – pâturage, puis pertes de stockage, puis export stocké) et le compare aux épandages **déclarés** dans la carte ferme : un écart supérieur à 25 % produit un **diagnostic** (pas une erreur — l'import de fumier extérieur est légitime). Le module sol ne compte pas le

N déposé au pâturage (EF3PRP déjà compté côté fumier — aucun double comptage).

Sortie : `results.json` — une entrée par simulation. L'entrée de la grille appariée (`sim_id = mc_cas_enteric_paired`) contient, sous `uncertainty` :

- `emissions_table` — une ligne par itération : `iteration`, une colonne `<variante>__<indicateur>` par variante (`indicateurs = gwp100, gwp20, gwpstar, ch4_kg, co2_kg, n2o_kg` — totaux ferme), et les colonnes par **source × gaz de la variante principale** (`enteric__ch4_kg, soil_n2o__n2o_kg`, etc. — liste dans `source_gas_columns`) ;
- `parameter_draws_table` — la valeur tirée de **chaque** paramètre à chaque itération (traçabilité complète) ;
- `central_gwp_factors` — facteurs GWP centraux (table 7.15 AR6), utilisés par l'analyse R pour séparer erreur d'inventaire et erreur de caractérisation ;
- `farm_indicators_stats, paired_differences_stats` — statistiques par variante et différences appariées contre la variante principale.

Cycle de vie de `results.json` : chaque exécution démarre sur un fichier vierge ; un `results.json` précédent est déplacé vers `results_archive/results_<horodatage>.json` (aucune perte, ISO 14044 §4.5 ; les figures R ne mélangent jamais deux exécutions). Le répertoire d'archive est ignoré par git, comme `results.json`.

Étape 3 — Analyses R

Toutes les figures et CSV vont dans `R/` par défaut (ou le répertoire passé en second argument). Les scripts lisent `results.json` — régénéré après chaque modification des cartes.

3.1 — Analyse de sensibilité appariée

Rscript `R/analyse_enteric_sensitivity.R results.json`

Quatre sorties complémentaires :

1. **Corrélations paramètres** → **GWP100** —

`enteric_paired_correlation.csv` (Pearson + Spearman, par variante) et `heatmap_fig_correlation_parametres_ch4.png`. Deux panneaux : erreurs d'**inventaire** (kg de gaz émis) vs erreurs de **caractérisation** (facteurs GWP).

2. **Effet pur du choix de modèle** —

`enteric_paired_model_effect.csv` et

fig_effet_modeles_gwp100.png : différence appariée de chaque alternative contre la variante principale. Deux barres d'erreur par alternative : erreur **totale** (facteurs GWP tirés) et erreur **inventaire seulement** (facteurs centraux).

3. Décomposition de variance du GWP100 —

enteric_variance_decomposition.csv et fig_decomposition_variance_gwp100.png. Décomposition **exacte** par covariance : chaque terme reçoit $\text{Cov}(\text{terme}, \text{GWP100_total}) / \text{Var}(\text{GWP100_total})$; les contributions somment à 100 % (aucune approximation, aucune hypothèse d'indépendance).

4. Tables brutes — enteric_paired_emissions.csv et

enteric_paired_parameter_draws.csv, pour toute analyse ad hoc.

3.2 — Figures descriptives par variante

Script	Figure	Contenu
plot_enteric_ch4_groups.R	fig_ch4_par_lot.png	CH ₄ entérique par lot (mean ± sd, p5–p95), par variante
plot_enteric_ch4_by_model.R	fig_ch4_par_modele.png	CH ₄ entérique ferme par variante
plot_enteric_ch4_per_head_day.R	fig_ch4_par_tete_jour.png	émissions par tête et par jour (comparaison inter-modèles)
plot_manure_ch4_by_model.R	fig_ch4_fumier_par_modele.png	CH ₄ fumier par système de gestion, par variante
plot_ration_comparison_correlation.R	fig_correlation_rations.png	corrélation équations IPCC vs rations mesurées

Étape 4 — Tests et contrat Python → R

```
python -m pytest tests/ # 137 tests
```

- **Tests unitaires** (tests/test_engine.py, tests/test_farm_spec.py, tests/test_card.py, ...) : cohérence des équations (re-calcul indépendant des formules IPCC/Tier-3), dérivation du pâturage et du fumier disponible, routage du N déposé, exports frais/stocké (tests **physiques** : l'export frais supprime les émissions de stockage), validation des cartes (fail-fast), idempotence des tirages, appariement des itérations.

- **Contrat Python → R** (tests/test_r_contract.py) : les scripts R lisent results.json via des clés implicites ; une clé renommée casserait les figures **silencieusement**. Ce fichier exécute un cas d'étude complet puis vérifie que **chaque chemin de clé** déréférencé par les scripts R existe avec la structure attendue. Si vous ajoutez une sortie consommée par R, ajoutez-y le test correspondant.

Étape 5 — Documentation auto-générée

```
pdoc pblca -o docs_html      # génère docs_html/ (HTML statique)
pdoc pblca                   # sert la doc localement (navigateur)
```

Ouvrir `docs_html/index.html` ou

`docs_html/pblca.html`. Liens utiles :

- `LCAEngine` — orchestration :

`run`,

`run_monte_carlo`,

`run_ration_comparison`,

`run_paired_variant_grid`

- `load_card` /

`run_card` — cartes TOML (validation,
exécution, traçabilité)

- `FarmSpec`,

`GrazingEventSpec`,

`SyntheticFertilisationSpec`,

`OrganicFertilisationSpec`

— spécification déclarative de la ferme

- `ParameterSet` — paramètres tracés :

`draw`,

`central_values`

- `GasLedger` — inventaire traçable :

`add`,

`total`,

`total_by_source`

- `characterize` — impacts (couche 3) ;

`GwpStarInputs`,

`compute_gwpstar`

- `SimulationResult` — résultat

d'une simulation (impacts, ledger, traces par slot)

- `DataStore` — stockage JSON et archivage

- `ModelSpec` — fiche d'une variante

(slot, tier, référence, champs requis)

Règle du dépôt : `pytest` et `pdoc pblca -o docs_html` doivent passer avant chaque commit (`AGENTS.md`). Après toute modification de ce guide, régénérer le PDF : `python scripts/make_guide_pdf.py` (`GUIDE.md` reste la source de vérité). Les docstrings étant en anglais (convention du dépôt), la

doc générée est en anglais ; ce guide, lui, est en français.

Pour aller plus loin (niveau développeur)

Mécanisme des tirages appariés (`pblca/mc.py`, `run_paired_variant_grid`). À chaque itération :

1. `engine.params.draw(rng)` tire **toutes** les valeurs de paramètres (y compris les facteurs GWP) ;
2. les rations mesurées sont perturbées (un facteur log-normal par groupe appliqué à DMI et GE, un par méthode de CH₄ mesuré) — *les mêmes mesures perturbées pour toutes les variantes*, donc l'appariement tient aussi pour les variantes `*_measured_ingestion` et `measured_ahcs` ;
3. chaque variante est évaluée avec ce tirage commun et ses indicateurs remplissent la ligne.

Conséquence : `Var(colonne_A - colonne_B)` ne contient que l'effet du choix de modèle — d'où la figure d'effet-modèle et les différences appariées de l'étape 3.1.

Chaînage fumier → **sol** (`pblca/processes/manure.py`, `pblca/processes/soil.py`). La part pâturage de chaque lot est dérivée des événements de pâturage de la carte ferme (`grazing = Σ jours / 365`) ; le N déposé est routé vers chaque parcelle pâturée (`n_excreta_grazing`, kg N/ha/an) ; le fumier stocké disponible résulte des excretions logées, des pertes de stockage (volatilisation, lessivage) et des exports (frais avant stockage, stocké après) ; les épandages déclarés dans la carte sont comparés au disponible (diagnostic > 25 %). Le module `sol` consomme les apports déclarés (minéral + organique) sans jamais recompter le N pâturé (EF3PRP côté fumier).

Paramètres de caractérisation (`pblca/params.py`) : les facteurs GWP sont rentrés comme les **valeurs tabulées de la table 7.15 AR6** (valeur centrale ± écart-type), convertis en paramètres log-normaux *dans le code* — on ne rentre jamais les paramètres μ/σ dérivés. Règle du dépôt : aucune valeur interpolée quand la source fournit une valeur tabulée.

Ajouter une variante : déclarer le modèle dans le slot concerné (`pblca/processes/...`, enregistré dans `pblca/registry.py` avec sa `ModelSpec` : référence, champs requis), puis l'ajouter à `[variant_grid]` dans la carte étude. Elle sera automatiquement balayée, appariée et analysée ; si elle exige des champs de groupe absents de la ferme, elle est exclue avec la raison plutôt que d'échouer.

Ajouter une ferme : écrire une carte ferme dans `cards/farms/` — elle est immédiatement référençable par `farm = "<nom>"` dans toute carte étude (pblca farms la liste). Les clés des deux cartes étant validées au chargement, une faute de frappe échoue immédiatement avec la liste des valeurs acceptées.

Ajouter une analyse R : vérifier les clés consommées dans `tests/test_r_contract.py` (et y ajouter les nouvelles), documenter les sorties dans `R/README.md`, et respecter la convention `sim_id = mc_<cas>_<slot>_<variante>` pour le regroupement des entrées.